

Politechnika Wrocławska

Systemy wbudowane

Sprawozdanie - Python i MicroPython – środowisko, emulator, kod przenośny

https://github.com/melamann00/systemy_wbudowane/tree/main/proj1

Jan Nowak

284138

Filip Bassa

284183

Spis treści

1 Wstęp i opis problemu **2**

1.1 Wzór matematyczny – wyprowadzenie krok po kroku 2

2 Środowisko **3**

3 Realizacja **3**

3.1 Struktura programu 3

3.2 Zastosowanie metody pierwiastka Newtona 4

3.3 Opis testów 4

4 Testy i prezentacja działania **4**

4.1 Wynik testów i trzy przykłady 4

4.2 Pomiar czasu 5

4.3 Sesja interaktywna i obsługa błędów 5

5 Przeniesienie na ESP8266 **5**

6 Wnioski **6**

1 Wstęp i opis problemu

Zadaniem było napisanie przenośnego (PC $\leftrightarrow$ ESP8266) programu `funkcja.py` realizującego prostą funkcję matematyczną. Numer wariantu wynika ze wzoru (numer na liście obecności $-$ 1) mod 12 + 1; w moim przypadku jest to **wariant 8 – pierwiastek kwadratowy metodą Newtona (Herona)**, z porównaniem do `math.sqrt`.

Program zawiera: (a) funkcję `pierwiastek_newton(a)` zwracającą wynik bez `input()`/`print()`, (b) funkcję `testy()` z siedmioma asercjami (przypadki brzegowe, typowe, duże i małe argumenty), (c) funkcję `main()` wyświetlającą wyniki dla trzech przykładów, mierzącą czas wykonania (`time.ticks_ms()`, `time.ticks_diff()`) i przyjmującą dane od użytkownika z obsługą błędów.

1.1 Wzór matematyczny – wyprowadzenie krok po kroku

Szukamy $x = \sqrt{a}$ dla $a > 0$, czyli miejsca zerowego funkcji

$$f(x) = x^2 - a, \qquad f'(x) = 2x. \tag{1}$$

Metoda Newtona zastępuje f w pobliżu przybliżenia x_n styczną i bierze jej miejsce zerowe:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} = x_n - \frac{x_n^2 - a}{2x_n} = \frac{2x_n^2 - x_n^2 + a}{2x_n} = \frac{1}{2}\left(x_n + \frac{a}{x_n}\right). \tag{2}$$

Jest to wzór Herona: nowe przybliżenie jest średnią arytmetyczną x_n i a/x_n. Jeśli $x_n > \sqrt{a}$, to $a/x_n < \sqrt{a}$, więc liczby te leżą po przeciwnych stronach pierwiastka.

Zbieżność. Oznaczmy błąd $e_n = x_n - \sqrt{a}$. Wtedy

$$x_{n+1} - \sqrt{a} = \frac{x_n^2 - 2x_n\sqrt{a} + a}{2x_n} = \frac{(x_n - \sqrt{a})^2}{2x_n} \quad \Rightarrow \quad e_{n+1} = \frac{e_n^2}{2x_n}. \tag{3}$$

Błąd jest więc *podnoszony do kwadratu* (zbieżność kwadratowa: liczba poprawnych cyfr mniej więcej podwaja się w każdym kroku). Ponadto $e_{n+1} \geq 0$ dla każdego $x_n > 0$, więc po pierwszym kroku ciąg maleje monotonicznie do $\sqrt{a}$ od góry.

Kryterium zatrzymania. Iteracje kończę, gdy względna zmiana jest mała: $|x_{n+1} - x_n| \leq \mathrm{TOL} \cdot x_{n+1}$, z limitem `MAX_ITER`$= 100$ jako zabezpieczeniem.

Punkt startowy. W programie przyjęto $x_0 = a$ dla $a \geq 1$ oraz $x_0 = 1$ dla $a < 1$. Jest to prosty wybór, który zawsze leży powyżej (lub równo z) $\sqrt{a}$ dla $a \geq 1$; dla $a < 1$ mamy $\sqrt{a} < 1 = x_0$. Dla $x_n \gg \sqrt{a}$ krok daje $x_{n+1} \approx x_n/2$, więc dopóki przybliżenie jest dalekie od wyniku, błąd maleje tylko o połowę na iterację. Liczba iteracji rośnie zatem w przybliżeniu jak $\log_2 \sqrt{a}$ (faza „dojścia") plus kilka kroków szybkiej zbieżności kwadratowej – widać to w pomiarach (rozdz. 4: 6 iteracji dla $a = 2$, 25 dla 10^{12}, 55 dla 10^{30}).

Przykład ręczny: $a = 2$ $\quad (x_0 = a = 2)$:

n	x_n	$\lvert x_n - \sqrt{2}\rvert$
0	2	$5{,}9 \cdot 10^{-1}$
1	1,5	$8{,}6 \cdot 10^{-2}$
2	1,416 666 667	$2{,}5 \cdot 10^{-3}$
3	1,414 215 686	$2{,}1 \cdot 10^{-6}$
4	1,414 213 562 375	$1{,}6 \cdot 10^{-12}$
5	1,414 213 562 373 095	$2{,}2 \cdot 10^{-16}$

Tabela 1: Kolejne przybliżenia $\sqrt{2}$ – widać podwajanie liczby poprawnych cyfr (błędy $10^{-1} \to 10^{-2} \to 10^{-3} \to 10^{-6} \to 10^{-12}$). Krok 6 potwierdza zbieżność (zmiana = 0).

2 Środowisko

- system operacyjny: Linux 6,
- wersja Pythona: 3.14.8 (`python --version`),
- IDE: Neovim,
- emulator: `machine.py` z pakietu startowego,
- użyte moduły: `machine`, `time`, `math` (wszystkie dostępne w MicroPythonie),
- platforma docelowa: ESP8266 z MicroPythonem

3 Realizacja

3.1 Struktura programu

element	rola
`TOL, MAX_ITER`	względna tolerancja zbieżności
`pierwiastek_newton(a)`	logika: walidacja ($a < 0$), punkt startowy, pętla Newtona; zwraca `(wynik, iteracje)`
`testy()`	testy automatyczne
`zmierz(n)`	pomiar czasu `ticks_ms`/`ticks_diff` dla Newtona i `math.sqrt`
`main()`	demonstracja na 3 przykładach, pomiar, pętla z danymi od użytkownika i obsługą błędów

Zgodnie z regułą nr 1 z instrukcji obliczenia i walidacja są oddzielone od obsługi konsoli. Użyto tylko `str.format()`, `math`, `time` i prostych typów.

3.2 Zastosowanie metody pierwiastka Newtona

```
def pierwiastek_newton(a, tol=TOL):
    if a < 0:
        raise ValueError("liczba nie moze byc ujemna")
    if a == 0:
        return 0.0, 0
    if a >= 1:
        x = a
    else:
        x = 1.0
    for i in range(1, MAX_ITER + 1):
        nowy = 0.5 * (x + a / x)
        if abs(nowy - x) <= tol * nowy:
            return nowy, i
        x = nowy
    return x, MAX_ITER
```

3.3 Opis testów

Funkcja `testy()` zawiera 7 testów i na końcu wypisuje „Testy OK”. Wyniki zmiennoprzecinkowe porównuję z tolerancją `abs(a - b) < ...`.

dane	cel testu
$a = 0$	a/x byłoby dzieleniem przez zero, więc funkcja zwraca 0 bez iteracji
$a = 1$	pierwiastek równy argumentowi
$a = 4$	kwadrat liczby całkowitej
$a = 2$	liczba niewymierna, porównanie z 1,41421356237
$a = 0{,}25$	argument < 1 – sprawdza drugą gałąź punktu startowego
$a = 10^{12}$	duży argument, wynik 10^6
$a = 123{,}456$	wartość „losowa”, porównanie z `math.sqrt`

Obsługa błędnych danych znajduje się w `main()`: wartość ujemna powoduje `ValueError` z funkcji, a tekst niebędący liczbą – `ValueError` z `float()`. Przecinek dziesiętny jest zamieniany na kropkę.

4 Testy i prezentacja działania

4.1 Wynik testów i trzy przykłady

```
❯ py funkcja1.py
Testy OK
sqrt(2) = 1.4142135624 (Newton, 6 iteracji), math.sqrt = 1.4142135624
sqrt(144) = 12.0000000000 (Newton, 9 iteracji), math.sqrt = 12.0000000000
sqrt(0.01) = 0.1000000000 (Newton, 9 iteracji), math.sqrt = 0.1000000000
```

Rysunek 1: Wynik `testy()` oraz trzy przykłady obliczeń z `main()`: Newton daje taki sam wynik jak `math.sqrt` po 5–6 iteracjach.

4.2 Pomiar czasu



Rysunek 2: Pomiar dla $n = 20000$ pierwiastków (kolejne liczby $1, 2, \ldots, 20000$).

`math.sqrt` jest szybszy, bo wykonuje się jako jedna instrukcja procesora, a nie jako pętla interpretowana w Pythonie. Dla małego n rozdzielczość jest porównywalna z mierzonym czasem, dlatego dla $n = 20000$ czas `math.sqrt` wyszedł 0 ms i iloraz jest nieokreślony.

4.3 Sesja interaktywna i obsługa błędów

Różnica względem `math.sqrt` wynosi co najwyżej o precyzję `float64`. Liczba iteracji rośnie wraz z wielkością argumentu.

```
❯ py funkcja1.py
Testy OK
sqrt(2) = 1.4142135624 (Newton, 6 iteracji), math.sqrt = 1.4142135624
sqrt(144) = 12.0000000000 (Newton, 9 iteracji), math.sqrt = 12.0000000000
sqrt(0.01) = 0.1000000000 (Newton, 9 iteracji), math.sqrt = 0.1000000000
20000 pierwiastkow: Newton 29 ms, math.sqrt 1 ms
Podaj liczbe (q - koniec): 25
Newton: 5.0000000000 (8 iteracji), math.sqrt: 5.0000000000, roznica: 0.00e+00
Podaj liczbe (q - koniec): 2,5
Newton: 1.5811388301 (6 iteracji), math.sqrt: 1.5811388301, roznica: 2.22e-16
Podaj liczbe (q - koniec): -4
Blad: liczba nie moze byc ujemna
Podaj liczbe (q - koniec): abc
Blad: could not convert string to float: 'abc'
Podaj liczbe (q - koniec):
Blad: could not convert string to float: ''
Podaj liczbe (q - koniec): 1e30
Newton: 1000000000000000.0000000000 (55 iteracji), math.sqrt: 1000000000000000.0000000000, roznica: 0.00e+00
Podaj liczbe (q - koniec): q
```

Rysunek 3: Dane od użytkownika: poprawne, ujemna, tekst, pusty ciąg i bardzo duża liczba. Błędne dane nie kończą programu tracebackiem; zakończenie klawiszem `q`

5 Przeniesienie na ESP8266

Program nie używa sprzętu, więc wystarczy wgrać `funkcja.py` jako `main.py`. Na płytce `import machine` jest wbudowany, a `input()`/`print()` działają w konsoli REPL. Spodziewane różnice :

- **Precyzja liczb.** Port ESP8266 używa 32-bitowego `float`, a PC 64-bitowego. Symulacja algorytmu w arytmetyce `float32` pokazała, że kryterium z TOL $= 10^{-12}$ nadal się spełnia, ale asercje z tolerancją 10^{-9} by zawiodły: błąd $\sqrt{2}$ w `float32` wynosi ok. $2{,}4 \cdot 10^{-8}$, a $\sqrt{123{,}456}$ ok. $1{,}5 \cdot 10^{-7}$. Na urządzeniu trzeba by zmienić tolerancje w `testy()` do ok. 10^{-5}.
- **Zakres.** Maksimum `float32` to $\approx 3{,}4 \cdot 10^{38}$.

- **Czas.** Procesor 80 MHz i interpretacja kodu są wielokrotnie wolniejsze niż PC. Przewidujemy, że pomiar $n = 20000$ potrwa od sekund do kilkunastu, a `math.sqrt` nadal będzie wyraźnie szybszy.
- **Pamięć.** Program nie tworzy dużych list, więc zużycie RAM jest znikome.
- **Rozdzielczość czasu** `ticks_ms` jest taka sama (1 ms), licznik „przekręca się", dlatego różnicę liczę zawsze przez `ticks_diff`.

6 Wnioski

- Metoda Newtona–Herona daje wynik zgodny z `math.sqrt` w kilk iteracjach, co potwierdza zbieżność kwadratową $e_{n+1} = e_n^2/(2x_n)$ (tab. 1).
- Własna implementacja jest ok. 30 razy wolniejsza niż `math.sqrt`.
- **Ograniczenie 1 – punkt startowy** $x_0 = a$**.** Liczba iteracji rośnie z $\log_2 \sqrt{a}$. Dla $a = 10^{300}$ pętla kończy się na `MAX_ITER`=100 bez zbieżności i zwraca błędny wynik, co zostało sprawdzone osobno.
- **Ograniczenie 2 – NaN i** ∞**.** Wpisanie `nan` lub `inf` nie powoduje błędu programu, ale funkcja wykonuje 100 iteracji i zwraca NaN. Wystarczy dodać jawne odrzucenie takich wartości.
- **Ograniczenie 3 – testy.** `testy()` nie zawiera asercji dla błędnych danych, sprawdzenie to pokazuję tylko w sesji interaktywnej. Zamknięcie wejścia daje `EOFError`.
- **Ograniczenie 4 – przenośność precyzji**: tolerancje testów dobrane dla `float64` wymagają zmiany na ESP8266.